

Assignment 4: Data Wrangling

Tracy Che

OVERVIEW

In this assignment, you will build a reproducible workflow that cleans four EPA air-quality files, combines them, reshapes the result, and produces grouped summaries. Date conversion is a brief review from Module 3; the emphasis is on data wrangling.

Directions

1. Rename this file `<FirstLast>_A04_DataWrangling.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. Ensure that code in code chunks does not extend off the page in the PDF.

Set up your session

Questions 1–2

- 1a. Load the `tidyverse` and `here` packages into your session.
 - 1b. Check your working directory.
 - 1c. Read all four raw files associated with the EPA Air dataset. See the README file for information about the variables and measurements.
2. Add the appropriate code to reveal the dimensions of the four datasets.

```
#1a
library(tidyverse)
library(here)
```

```
#1b
getwd()
```

```
## [1] "/home/guest/tracyche"
```

```
here::i_am("Assignments/TracyChe_A04_DataWrangling.Rmd")
here()
```

```
## [1] "/home/guest/tracyche"
```

```
#1c
```

```
pm25_2024 <- read.csv(  
  here("Data/A4/EPAair_PM25_NC2024_raw.csv")  
)
```

```
pm25_2025 <- read.csv(  
  here("Data/A4/EPAair_PM25_NC2025_raw.csv")  
)
```

```
o3_2024 <- read.csv(  
  here("Data/A4/EPAair_O3_NC2024_raw.csv")  
)
```

```
o3_2025 <- read.csv(  
  here("Data/A4/EPAair_O3_NC2025_raw.csv")  
)
```

```
#2
```

```
dim(pm25_2024)
```

```
## [1] 9752 22
```

```
dim(pm25_2025)
```

```
## [1] 10184 22
```

```
dim(o3_2024)
```

```
## [1] 9519 21
```

```
dim(o3_2025)
```

```
## [1] 9466 21
```

All four datasets should have the same number of columns (PM2.5 has one extra column), but unique record counts (rows). Do your datasets follow this pattern? Yes. The two PM2.5 datasets have 22 columns, while the two ozone datasets have 21 columns.

Wrangle individual datasets to create processed files.

Questions 3–5

3. Change the Date columns to be date objects.
4. Select the following columns: DDate, Daily.AQI.Value, Local.Site.Name, AQS.Parameter.Description, County, Site.Latitude, Site.Longitude

5. For the PM2.5 datasets, fill all cells in AQS.Parameter.Description with “PM2.5” (all cells in this column should be identical).

#3

```
pm25_2024$Date <- mdy(pm25_2024$Date)
pm25_2025$Date <- mdy(pm25_2025$Date)

o3_2024$Date <- mdy(o3_2024$Date)
o3_2025$Date <- mdy(o3_2025$Date)
```

check

```
class(pm25_2024$Date)
```

```
## [1] "Date"
```

#4

```
pm25_2024 <- select(
  pm25_2024,
  Date,
  Daily.AQI.Value,
  Local.Site.Name,
  AQS.Parameter.Description,
  County,
  Site.Latitude,
  Site.Longitude
)
```

```
pm25_2025 <- select(
  pm25_2025,
  Date,
  Daily.AQI.Value,
  Local.Site.Name,
  AQS.Parameter.Description,
  County,
  Site.Latitude,
  Site.Longitude
)
```

```
o3_2024 <- select(
  o3_2024,
  Date,
  Daily.AQI.Value,
  Local.Site.Name,
  AQS.Parameter.Description,
  County,
  Site.Latitude,
  Site.Longitude
)
```

```
o3_2025 <- select(
```

```

o3_2025,
Date,
Daily.AQI.Value,
Local.Site.Name,
AQS.Parameter.Description,
County,
Site.Latitude,
Site.Longitude
)

#5

pm25_2024 <- mutate(
  pm25_2024,
  AQS.Parameter.Description = "PM2.5"
)

pm25_2025 <- mutate(
  pm25_2025,
  AQS.Parameter.Description = "PM2.5"
)

```

Question 6

6. Save all four processed datasets in the Processed folder. Use the same file names as the raw files but replace “raw” with “processed”.

```

#6

write.csv(
  pm25_2024,
  row.names = FALSE,
  file = here(
    "Data/A4/Processed/EPAair_PM25_NC2024_processed.csv"
  )
)

write.csv(
  pm25_2025,
  row.names = FALSE,
  file = here(
    "Data/A4/Processed/EPAair_PM25_NC2025_processed.csv"
  )
)

write.csv(
  o3_2024,
  row.names = FALSE,
  file = here(
    "Data/A4/Processed/EPAair_O3_NC2024_processed.csv"
  )
)

```

```
write.csv(
  o3_2025,
  row.names = FALSE,
  file = here(
    "Data/A4/Processed/EPAair_03_NC2025_processed.csv"
  )
)
```

Combine datasets

Question 7

7. Stack the four datasets with `bind_rows()`. Check that their column names are identical first.

#7

#checking column names
`colnames(pm25_2024)`

```
## [1] "Date"                "Daily.AQI.Value"
## [3] "Local.Site.Name"     "AQS.Parameter.Description"
## [5] "County"              "Site.Latitude"
## [7] "Site.Longitude"
```

`colnames(pm25_2025)`

```
## [1] "Date"                "Daily.AQI.Value"
## [3] "Local.Site.Name"     "AQS.Parameter.Description"
## [5] "County"              "Site.Latitude"
## [7] "Site.Longitude"
```

`colnames(o3_2024)`

```
## [1] "Date"                "Daily.AQI.Value"
## [3] "Local.Site.Name"     "AQS.Parameter.Description"
## [5] "County"              "Site.Latitude"
## [7] "Site.Longitude"
```

`colnames(o3_2025)`

```
## [1] "Date"                "Daily.AQI.Value"
## [3] "Local.Site.Name"     "AQS.Parameter.Description"
## [5] "County"              "Site.Latitude"
## [7] "Site.Longitude"
```

#stack the rows

```
EPA.air.all <- bind_rows(
  pm25_2024,
  pm25_2025,
```

```
o3_2024,
o3_2025
)

dim(EPA.air.all)
```

```
## [1] 38921      7
```

Question 8

8. Use one pipeline (`%>%`) to create a daily site-level dataset that meets the following conditions:

- Include only sites that the four data frames have in common:

“Linville Falls”, “Durham Armory”, “Leggett”, “Hattie Avenue”,
 “Clemmons Middle”, “Mendenhall School”, “Frying Pan Mountain”, “West Johnston Co.”, “Garinger High School”, “Castle Hayne”, “Pitt Agri. Center”, “Bryson City”, “Millbrook School”

(the function `intersect` can figure out common factor levels - but it will include sites with missing site information, which you don’t want...)

- Some sites have multiple measurements per day. Group by date, site name, AQS parameter, and county, then calculate the mean AQI value, latitude, and longitude. Handle missing values explicitly in each mean.
- Add numeric `Month` and `Year` columns from `Date`. Check if results are integers.
- Hint: the dimensions of this dataset should be $15,012 \times 9$.

```
#8

EPA.air.daily <-
EPA.air.all %>%
  filter(
    Local.Site.Name %in% c(
      "Linville Falls",
      "Durham Armory",
      "Leggett",
      "Hattie Avenue",
      "Clemmons Middle",
      "Mendenhall School",
      "Frying Pan Mountain",
      "West Johnston Co.",
      "Garinger High School",
      "Castle Hayne",
      "Pitt Agri. Center",
      "Bryson City",
      "Millbrook School"
    )
  ) %>%
  group_by(
    Date,
```

```

    Local.Site.Name,
    AQS.Parameter.Description,
    County
  ) %>%
  summarise(
    Daily.AQI.Value = mean(
      Daily.AQI.Value,
      na.rm = TRUE
    ),
    Site.Latitude = mean(
      Site.Latitude,
      na.rm = TRUE
    ),
    Site.Longitude = mean(
      Site.Longitude,
      na.rm = TRUE
    ),
    .groups = "drop"
  ) %>%
  mutate(
    Month = month(Date),
    Year = year(Date)
  )

#check
dim(EPA.air.daily)

```

```
## [1] 15012      9
```

Questions 9–10

9. Use `pivot_wider()` so that ozone and PM2.5 AQI values occupy separate columns. Each location-date combination should occupy one row.
10. Call up the dimensions of your new tidy dataset.

```

#9

EPA.air.wide <-
  EPA.air.daily %>%
  pivot_wider(
    names_from = AQS.Parameter.Description,
    values_from = Daily.AQI.Value
  )

#10

dim(EPA.air.wide)

```

```
## [1] 9037      9
```

Question 11

11. Save your processed dataset with the following file name: “EPAair_O3_PM25_NC2425_Processed.csv”

```
#11

write.csv(
  EPA.air.wide,
  row.names = FALSE,
  file = here(
    "Data/A4/Processed/EPAair_O3_PM25_NC2425_Processed.csv"
  )
)
```

Generate summary tables

Questions 12–13

12. Create a monthly summary data frame grouped by site, month, and year. Calculate mean ozone and PM2.5 AQI for each group, handling missing values explicitly. Then use `drop_na()` to remove rows only when mean **ozone** is missing. Mean PM2.5 may remain missing.
13. Call up the dimensions of the summary dataset.

```
#12

EPA.air.monthly.all <-
  EPA.air.wide %>%
  group_by(
    Local.Site.Name,
    Month,
    Year
  ) %>%
  summarise(
    mean_ozone = mean(
      Ozone,
      na.rm = TRUE
    ),
    mean_pm25 = mean(
      PM2.5,
      na.rm = TRUE
    ),
    .groups = "drop"
  )

EPA.air.monthly <-
  EPA.air.monthly.all %>%
  drop_na(mean_ozone)
```

#13

```
dim(EPA.air.monthly)
```

```
## [1] 238 5
```

Question 14

14. Why is `drop_na(mean_ozone)` more appropriate here than `na.omit()`? Test both approaches and compare the dimensions of the resulting summary data frames.

```
EPA.air.monthly.drop <-  
  EPA.air.monthly.all %>%  
  drop_na(mean_ozone)
```

```
EPA.air.monthly.omit <- na.omit(  
  EPA.air.monthly.all  
)
```

```
dim(EPA.air.monthly.drop)
```

```
## [1] 238 5
```

```
dim(EPA.air.monthly.omit)
```

```
## [1] 234 5
```

Answer: the ‘drop’ is more appropriate because it removes rows only when mean ozone is missing. It keeps rows with valid ozone values even when mean PM2.5 is missing. In contrast, ‘omit’ removes rows if any variable contains a missing value, so there is less rows left after ‘omit’.